Education and Information Technologies (2023) 28:12919–12943
https://doi.org/10.1007/s10639-023-11671-2

Effects of the interaction between metacognition teaching and students' learning achievement on students' computational thinking, critical thinking, and metacognition in collaborative programming learning

Wei Li[1,2] · **Cheng-Ye Liu**[1] · **Judy C. R. Tseng**[3]

Received: 19 September 2022 / Accepted: 14 February 2023 / Published online: 18 March 2023

Abstract

Collaborative programming can develop computational thinking and knowledge of computational programming. However, the researchers pointed out that because students often fail to mobilize metacognition to regulate and control their cognitive activities in a cooperation, this results in poor learning effects. Especially low-achieving students need more metacognitive support. Therefore, this study proposed a metacognition-based collaborative programming approach (M-CPA) to improve students' performance in collaborative programming. To evaluate the effectiveness of the method for students with different levels of learning achievement, a 7-week experiment was conducted. A total of 222 middle school students were divided into the experimental group with the M-CPA learning and the control group with the conventional collaborative programming approach (C-CPA). The results showed that learning methods and learning achievement had interactive effects on computational thinking tendency, critical thinking tendency, and metacognition tendency. M-CPA could significantly improve students' computational thinking tendency, critical thinking tendency, and metacognition tendency. Moreover, the proposed approach is more effective for low-achieving students. The results also showed that M-CPA could improve the students' achievement in program analysis questions.

Keywords Metacognition teaching · Collaborative programming · Learning achievement · Computational thinking tendency · Critical thinking tendency · Metacognition tendency

✉ Judy C. R. Tseng
judycrt@gmail.com

[1] STEM Education Research Center, Wenzhou University, Zhejiang Province, Wenzhou, China

[2] Ph.D. Program in Engineering Science, Chung Hua University, Hsinchu, Taiwan

[3] Department of Computer Science and Information Engineering, Chung Hua University, Hsinchu 300, Taiwan

1 Introduction

There has been a growing interest in learning programming (Grover & Pea, 2013; Kafai & Burke, 2013). With the widespread use of computers in social life, the cultivation of computational thinking ability has received increasing attention. Some researchers believe that children can develop computational thinking when they learn programming (Wong & Cheung, 2018). Because programming is not just coding, it requires applying computer science concepts such as abstraction and decomposition to solve problems (Lye & Koh, 2014). In addition, people use critical thinking skills, namely effective reasoning, systems thinking, and evidence evaluation (Binkley et al., 2012), to deal with computational problems. That is, learning to program not only develops computational thinking, but is also closely related to critical thinking, problem-solving, and more.

Although learning programming is essential, learning programming is often challenging and complex for most students (Robins et al., 2003; Rubio Sánchez et al., 2014). Because it involves a range of grammar knowledge and algorithmic skills implementation (Wang et al., 2012). Therefore, to improve students' learning performance in programming, Hanks et al. (2011) suggested incorporating collaborative learning into programming activities. Researchers believe that collaborative programming is helpful in developing CT skills and computational programming knowledge (Denner et al., 2014; Iskrenovic-Momcilovic, 2019), because collaborative programming can provide more opportunities for students to interact with their peers and realize shared knowledge construction (Altintas et al., 2014; Cai & Gu, 2019).

However, simply providing students with collaborative learning may not necessarily lead to good learning outcomes (Kreijns et al., 2003). Because collaborative learning not only requires the individual learning performance of the learners but also emphasizes the coordination and collaboration of group members when completing tasks (Cai & Gu, 2019). Some students will spend too much time discussing and arguing with their peers, resulting in less time and effort in learning tasks. In other words, students are often unable to use metacognition to regulate and control their cognitive learning activities during the process of cooperation (Barron, 2003; Hadwin & Oshige, 2011). Therefore, it is possible to provide learners with a metacognitive scaffold in collaborative learning, which can improve students' learning performance by guiding students to complete planning, monitoring, and reflection (Kwon et al., 2013).

In addition, different manifestations of metacognitive monitoring can be seen between high- and low-achieving students, with low-achieving students showing a lack of monitoring over their learning process (DiFrancesca et al., 2016). A study has found that learning strategies and metacognitive skills are the main factors contributing to the differences between high-achieving and low-achieving learners (Kinnunen & Vauras, 1995). Some teachers even believe that higher-order thinking activities are not suitable for low-achieving students (Zohar et al., 2001). They also believe that the cognitive demands of these tasks exceed the abilities of low-achieving students. These teachers believe that learning should progress from

simple, low-order cognitive tasks to higher-order thinking tasks. Low-achieving students tend to "get stuck" in low-order cognitive tasks that emphasize memory and practice, and they have difficulty progressing to higher-order thinking tasks (Zohar et al., 2001). Since programming is a hierarchical skill, students who do not understand one topic cannot move on to the next (Rahmat et al., 2012). This complex hierarchy of knowledge and skills increases the difficulty of learning programming (Linn & Dalbey, 1985), especially for low-achieving students. Therefore, it is necessary to investigate the different effects of metacognitive strategies on high- and low- achieving students.

Additionally, studies have found that high-achieving learners are more likely to develop critical thinking skills than low-achieving learners (Seventika et al., 2018). Early academic achievement has a long-term impact on subsequent academic performance. Therefore, early intervention should be performed to reduce the long-term impact of early low achievement (Rao et al., 2000). Although recent studies have applied metacognitive strategies to collaborative learning and have examined students' performance (Lai, 2021; Panadero & Järvelä, 2015), there are few studies that have focused on the different influences of these strategies on high-achieving and low-achieving students. To improve the effect of students' collaborative programming, this study proposes a metacognition-based collaborative programming approach (M-CPA) and designs an experiment to evaluate its effectiveness. Due to the importance of students' learning achievements in collaborative learning, this study explores whether there is an interaction between learning strategies and students' learning achievement in terms of students' computational thinking tendencies, critical thinking tendencies, and metacognitive tendencies. The list of research questions is as follows.

Q1: Does M-CPA improve students' programming learning achievement compared to the conventional collaborative programming approach (C-CPA) approach?
Q2: In programming collaborative learning activities, is there an interaction between learning methods and students' learning achievement in terms of computational thinking tendencies?
Q3: In programming collaborative learning activities, is there an interaction between learning methods and students' learning achievement in terms of critical thinking tendency?
Q4: In programming collaborative learning activities, is there an interaction between learning methods and students' learning achievement in terms of metacognition tendency?

2 Literature Review

2.1 Collaborative programming

In recent years, programming as a critical digital skill (Law et al., 2018) has attracted more and more attention from educators worldwide. Because programming

is not only an essential skill in computer science (Popat & Starkey, 2019), it can also promote students' computational thinking (Papadakis et al., 2016). Programming is a great way to create computational results and demonstrate computational thinking skills (Buitrago Flórez et al., 2017; Lye & Koh, 2014), and is a good way to develop computational thinking (Shute et al., 2017). Furthermore, programming can also develop students' critical thinking, algorithmic problem-solving skills, and other higher-order thinking skills (Scherer, 2016; Wong & Cheung, 2018).

However, many learners consider programming to be complex and challenging (Akinola, 2015). To motivate students to learn programming, researchers suggest using a collaborative approach (Hanks et al., 2011; Iskrenovic-Momcilovic, 2019). Collaborative learning in programming has been shown to have better learning outcomes compared to individual independent work (Goel & Kathuria, 2010; Iskrenovic-Momcilovic, 2019; Wei et al., 2021). Furthermore, collaborative programming can help improve students' computational thinking (Zhong et al., 2016). Social learning theory also emphasizes the importance of peers and participation in helping learners master the ability of computational thinking (Patarakin et al., 2019). In addition, programming requires students to achieve self-reflection through repeated trial and error, which promotes the development of their critical thinking skills (Hayes & Stewart, 2016; Psycharis & Kallia, 2017).

However, some researchers have pointed out that collaborative learning can lead to social slack (Demir & Seferoglu, 2020), and group members may even make less effort when working together on tasks than when learning alone, resulting in poor learning outcomes. In other words, if students lack guidance on collaborative programming, they may interfere with each other, which in turn affects learning outcomes.

2.2 The role of metacognition in collaborative learning

The concept of metacognition was first proposed by Flavell (1979). He believed that metacognition, on the one hand, refers to the individual's knowledge about his cognitive process, results, and any related things. On the other hand, it refers to the individual's active monitoring of his cognitive process, regulation of results, and coordination of various processes (Flavell, 1976, 1981). Metacognition is a high-level cognitive process. Baker and Brown (1984) divided metacognition into knowledge of cognition and regulation of cognition. Research has shown that students with high levels of metacognitive skills have an advantage in problem-solving (McCormick, 2003) and achieve higher levels of achievement (Cleary & Zimmerman, 2001). Metacognition is an essential feature of self-regulated learning, which coordinates learning by planning, monitoring, and evaluating cognitive processes (Schraw & Moshman, 1995).

Metacognition is crucial in collaborative learning (Järvelä et al., 2016). On the one hand, learners must use metacognitive coordination and collaborative processes between peers, such as negotiating group task goals and plans, monitoring group progress, and reflecting on collaborative learning outcomes. On the other hand, learners also need to plan, monitor and reflect on their own learning goals,

engagement, and task completion (Jeong & Hmelo-Silver, 2016). However, due to insufficient metacognitive abilities of some students (Cho & Jonassen, 2002), when solving complex problems, they are unable to choose appropriate problem-solving strategies and fail to monitor and reflect on the learning process (Hadwin et al., 2011; Mayer, 1998, 2001), resulting in poor collaborative learning. Therefore, it is necessary to provide metacognitive support for students in collaborative learning activities, guide students to monitor their cognitive situation, and monitor and regulate the goals, process, and results of collaborative learning to make more reasonable decisions (Koriat et al., 2006). Furthermore, both low- and high-achieving individuals face difficulties in solving real-world tasks (Kramarski et al., 2001). In particular, some low-achieving students do not view the task as a whole but only focus on a part of the task (Lester, 1994). Researchers believe that explicit handling of metacognitive knowledge in teaching is more important for low-achieving students than high-achieving students (White & Frederiksen, 1998). Therefore, this study not only proposes a metacognition-based collaborative programming approach, but also aims to explore the impact of this approach on students with different learning achievements.

3 The metacognition-based collaborative programming system

The common element of metacognitive teaching is to conduct metacognitive regulation by guiding students to answer a series of metacognitive questions and to stimulate students' metacognition to better solve collaborative tasks (Kramarski et al., 2002). These metacognitive problems mainly include understanding the problems, constructing connections between old and new knowledge, formulating appropriate problem-solving strategies, and reflecting on the processes and solutions of problem-solving (Kramarski et al., 2002). Moreover, planning, monitoring, and evaluation are at the heart of metacognition (Dirkes, 1985; Flavell, 1979). To guide students in collaborative programming and stimulate their metacognition, this study designed a scaffolding for metacognitive problems based on those proposed by Kramarski et al. (2002) while taking the characteristics of collaborative programming into account. The problem is shown in Table 1.

Table 1 Metacognition problems scaffolding

Metacognition problems	Problem scaffolding
Comprehending problems and planning	What is the problem that must be solved by this programming task? What do you expect to achieve in this task?
Constructing connections	How is this problem/task different from what you have solved before?
Formulating strategies	How to solve this task? Can you describe the algorithm?
Reflection and Evaluation	Did your group solve today's programming task? Did you encounter difficulties in solving the problem? How is it solved? Is there any other solution to today's task?

Combined with the metacognition problem scaffolding, this research developed a metacognition-based collaborative programming system. The system architecture is shown in Fig. 1.

The system mainly includes a task allocation module, a collaborative learning module, and a metacognitive problem module. In addition, it also has a learning task database and a learning process database. After logging into the system, students can view the detailed information and requirements of the task through the task guide mechanism. After that, students will be randomly divided into study groups of 4–5 people and then they will discuss on the discussion board under the guidance of the metacognition problem module and collaborate to solve programming tasks. In the process of collaborative task solving, the metacognition problem module activates students' metacognition through question prompts. The first stage is the understanding of the problem. The purpose of this stage is for students to identify the problem to be solved and to try to describe the task in their own words. The next stage is to establish the connection between new and old knowledge, and guide students to pay attention to the similarities and differences between the tasks to be solved this time and the previous ones. For some students, it may be difficult to use old knowledge to assimilate new knowledge due to insufficient or weak initial knowledge reserves. Through problem guidance, help students mobilize the initial knowledge to solve new problems. After that, the metacognition problem module guides students to formulate strategies suitable for solving problems, describe algorithms in natural language or flow charts, compare the advantages and disadvantages of algorithms, and choose an algorithm to write a program, debug, and run. Finally, the system will guide learners to reflect on the problem-solving process and results, which will also help to adjust the next cooperation.

The system interface is shown in Fig. 2. When using the system to teach, teachers can decide whether to add metacognition problem modules. When the option

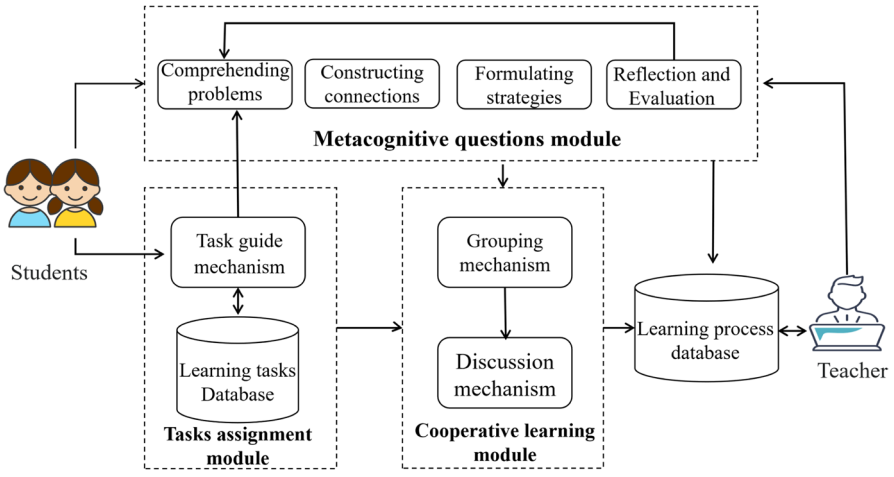

Fig. 1 System architecture

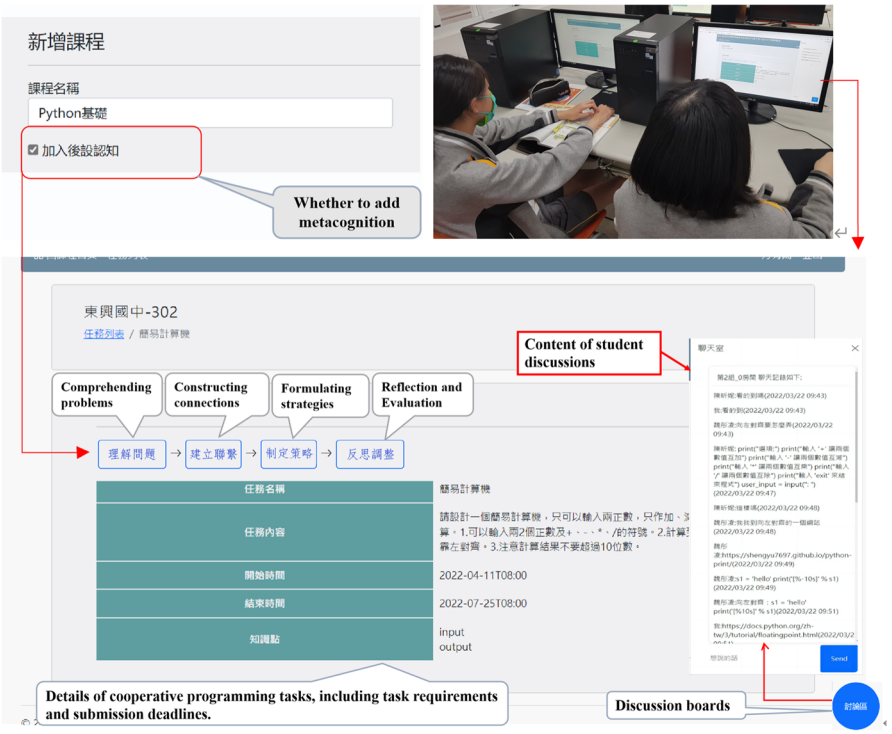

Fig. 2 Interface of metacognition-based collaborative programming system

"Add metacognition" is checked, the system will guide the students through the metacognition question module.

4 Methodology

4.1 Participants

Students in the third grade of a middle school participated in this experiment, and their average age was 15 years old. There were 222 students in 8 classes and the classes were randomly assigned to the experimental group and the control group. The experimental group adopted the metacognition-based collaborative programming approach (M-CPA), and the control group used the conventional computer-supported collaborative programming approach (C-CPA). There were 115 students in the experimental group, including 60 boys and 55 girls, who used M-CPA to study. In the control group, there were 107 students, including 52 boys and 55 girls, who adopted C-CPA. The two classes are taught by the same teacher with five years of teaching experience, and the teaching materials and learning tasks are the same.

4.2 Instrument

The measurement tools used in this study included a pre-test, post-test, computational thinking tendency, critical thinking tendency, and a group metacognition questionnaire. Both the pre-test and the post-test are multiple choice questions, with a total of 10 questions, each with 10 points, with a total score of 100 points. Seven of the questions are programming basic grammar questions, such as "Which of the following is a floating-point data type?". The basic grammar questions total 70 points. There are 3 program analysis questions, 30 points. Questions such as "Write the result of running the following program." Both the pre-test and post-test were developed by junior high school programming teachers with rich teaching experience. The purpose of the pre-test was to assess students' prior knowledge of programming, and the post-test was designed to evaluate their learning achievements.

The Cronbach's α of the computational thinking tendency, critical thinking tendency and metacognition tendency questionnaires were 0.84, 0.85 and 0.89, respectively, indicating that the questionnaires had a good level of reliability. The pre-questionnaire and post-questionnaire used a 5-point Likert type scale (5 = totally agree; 1 = totally disagree). As shown in Table 2.

4.3 Learning activities

The teacher assigned two collaborative programming tasks successively, namely, "Design A Simple Calculator" and "Design A Complex Calculator". Both groups were asked to complete the two tasks within five weeks. Descriptions and schedule of the two tasks are shown in Table 3.

4.4 Experimental Procedure

The experiment lasted 7 weeks, 45 min per week. Figure 3 presents the experimental procedure. In the first week, the teacher will introduce the use of the collaborative programming learning system to the students and ask the students to familiarize themselves with the operating interface. Students then complete the pre-test and pre-questionnaire. From week 2 to week 6, teachers introduce the basics of programming and then students work together to complete the programming tasks. Students are randomly divided into collaborative groups of 4 to 5 by the system. The experimental group used M-CPA, and the control group used C-CPA. Under the guidance of the metacognitive questions module, students in the experimental group clarified their understandings of the problem to be solved and established connections between old and new knowledge at the beginning of the collaboration. During the task completion process, the metacognitive questions module prompted messages reminding the students to develop solutions through discussions, to write the algorithm of the solutions and to implement the algorithm by programming. When the task was completed, the system guided the students to reflect on the problem-solving process and the results through prompts. Unlike the experimental group, conventional computer-supported collaborative learning was

Table 2 Questionnaire description

Name of the questionnaire	Questionnaire source	Questionnaire question example	Cronbach's α
Computational thinking tendencies	The questionnaire proposed by Hwang et al. (2020). There were 6 items	I can usually develop a step-by-step procedure for solving a complex problem	0.84
Critical thinking tendencies	The questionnaire was modified by Lin et al. (2019) based on the measure proposed by Chai et al. (2015). There were 6 items	During the learning process, I will evaluate different opinions to see which is more reasonable	0.84
Metacognition tendencies	The questionnaire was adapted from a measure developed by Lai and Hwang (2014) to measure learners' metacognitive abilities. There were 5 items in total	I ask myself how well I accomplished my goals once I'm finished	0.89

Table 3 Learning tasks and schedule

Task	Description	Schedule
Design a simple calculator	Students collaborate to design a simple calculator program in Python. The program allows the user to enter two positive numbers and perform addition and subtraction operations	**Week** 2–3
Design a complex calculator	Students collaborate to design a complex calculator program in Python. The program calculates the addition, subtraction, multiplication, division, and averaging of two numbers, as well as the square of one of the numbers	**Week** 4–6

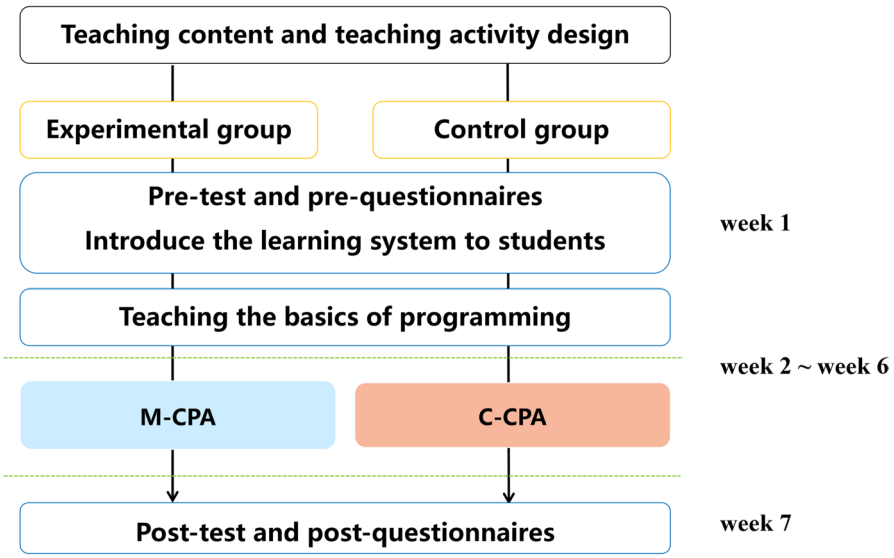

Fig. 3 Experimental design of the learning activities

adopted for students in the control group, without the guidance of the metacognitive questions module. After receiving the task assigned by the teacher, the students developed solutions through discussions, wrote the algorithm of the solutions and implemented the algorithm by programming to complete the task. In week 7, all the students completed the post-test and the post-questionnaire.

5 Results

This study developed a metacognition-based collaborative programming system to support students in collaborative learning. First, the influence of M-CPA on students' academic performance is discussed. Second, to further explore the interactive

effects of different learning methods and different learning achievements in computational thinking, metacognition, and critical thinking, a two-way ANCOVA method was adopted. According to some researchers (Elia et al., 2009; Ireson & Hallam, 2009), the high-achieving group included students who scored in the top 25% on the pre-test, while the low-achieving group included students in the bottom 25%. The independent variables were learning approach (M-CPA and C-CPA) and learning achievement (higher and lower).

5.1 Learning achievement

The one-way ANCOVA results of academic achievement are shown in Table 4. For basic grammar, the adjusted mean scores of the experimental group and the control group were M = 42.88 and M = 40.56, respectively. It can be seen that although the scores of the experimental group were higher, there was no significant difference in academic performance between the two groups ($F = 0.89$, $p = 0.35 > 0.05$). In terms of program analysis, adjusted mean scores for the experimental and control groups were M = 11.14 and M = 7.74, respectively. The academic performance of the experimental group was significantly higher than that of the control group ($F = 9.46$, $p = 0.002 < 0.01$), with a smaller effect size ($\eta^2 = 0.04 < 0.059$) (Cohen, 1988). The results show that the collaborative programming method based on metacognition has a better effect on improving students' program analysis ability.

5.2 Computational thinking tendency

The two-way ANCOVA was used to understand the influence of different learning methods and different learning achievements on students' computational thinking. The dependent variable was the post-test score of students' computational thinking tendency, and the covariate was the pre-test score of students' computational thinking tendency. Levene's test showed that the assumption of the homogeneity of variances was not rejected ($F = 1.46$, $p = 0.23 > 0.05$). Table 5 shows the results of the two-way ANCOVA. The results showed that the teaching methods had a significant effect on students' computational thinking tendency ($F = 5.27$, $p = 0.024 < 0.05$), and the effect size was 0.05. Regarding computational thinking tendency, there was a significant interaction between teaching methods and learning achievement

Table 4 Results of a one-way ANCOVA on students' learning achievement

Variable	Groups	*N*	Mean	*SD*	Adjusted Mean	Adjusted SD	*F*	η^2
Basic grammar	Experimental group	115	42.61	18.55	42.88	1.76	0.89	
	Control group	107	40.84	19.09	40.56	1.70		
Program analysis	Experimental group	115	11.04	9.21	11.14	0.77	9.46**	0.04
	Control group	107	7.85	7.53	7.74	0.80		

** $p < 0.01$

Table 5 The two-way ANCOVA result of computational thinking tendency

Variables	*SS*	df	*MS*	*F*	η^2
Pre-test	6.63	1	6.63	16.78***	0.13
Learning approaches	2.08	1	2.08	5.27*	0.05
Learning achievement	1.00	1	1.00	2.55	
Learning approaches*Learning achievement	2.02	1	1.83	5.10*	0.04
Error	43.86	111	0.40		

*$p<0.05$, ***$p<0.001$

($F=5.10$, $p=0.026<0.05$). Furthermore, the interaction effect ($\eta 2$) between the two was 0.02, representing a relatively small effect (Cohen, 1988).

To further investigate the results of the simple main effects analysis of students' learning achievement levels, the study used different teaching methods to examine students' computational thinking tendency. As shown in Table 6, the results showed a significant difference in the computational thinking tendency among students of different achievement levels in the experimental group ($F=4.12$, $p=0.048<0.05$). On the other hand, there was no significant difference between high and low-achieving students in the control group ($F=0.22$, $p=0.643>0.05$). From the results of the students studying through M-CPA, the level of academic performance has a significant impact on the tendency of computational thinking.

Table 7 shows the simple main effects analysis results of the influence of learning methods on the computational thinking tendency of students with different achievement levels. The results showed that low-achieving students learned with different methods, and there was a significant difference in computational thinking tendency ($F=8.87$, $p=0.004<0.01$). Although high-achieving students studied with different learning methods, there was no significant difference in computational thinking tendency ($F=0.01$, $p=0.94>0.01$). The results show that M-CPA is more beneficial to low-achieving students than to high-achieving students.

Figure 4 shows that different teaching methods and have an of learning achievement levels impact on students' computational thinking tendency. It can be seen in the figure that students who study with M-CPA have a higher

Table 6 Simple main-effect analysis results of learning achievement levels on students' computational thinking tendency

Variables		*SS*	df	*MS*	*F*	η^2
M-CPA (experimental group)	Between groups	1.79	1	1.79	4.12*	0.08
	Within groups	20.91	48	0.44		
	Total	22.85	50			
C-CPA (control group)	Between groups	0.07	1	0.07	0.22	
	Within groups	20.03	62	0.32		
	Total	30.26	64			

*$p<0.05$

Table 7 Simple main-effect analysis results of learning approach on students' computational thinking tendency

Variables		*SS*	df	*MS*	*F*	η^2
Higher achievement	Between groups	0.00	1	0.00	0.01	
	Within groups	19.24	57	0.34		
	Total	24.17	59			
Lower achievement	Between groups	4.07	1	4.07	8.87**	0.14
	Within groups	24.34	53	0.46		
	Total	30.86	55			

** $p < 0.01$

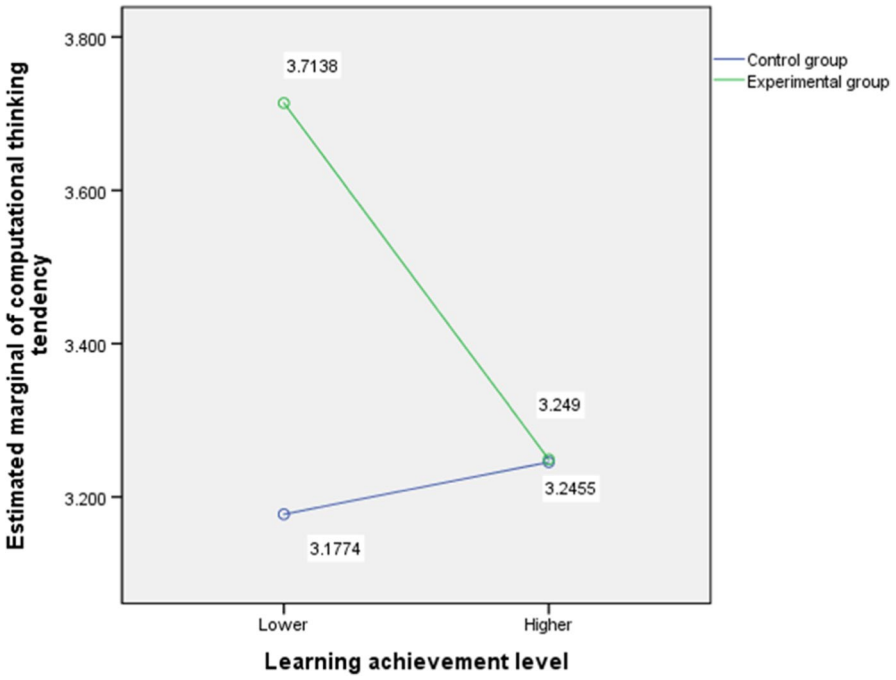

Fig. 4 Interaction in computational thinking tendency

computational thinking tendency. High-achieving students did not differ significantly in their computational thinking tendency, regardless of the collaborative programming method they used to learn. However, students with low achievement showed a significant improvement in computational thinking tendency when using M-CPA. This result suggests that M-CPA helps low-achieving students improve their computational thinking tendency.

5.3 Critical thinking tendency

The two-way ANCOVA was used to understand the interactive effects of different learning methods and different learning achievements on the students' critical thinking tendency. The dependent variable was the scores of students' c critical thinking tendency after the experiments, and the covariate was the scores of students' critical thinking tendency questionnaires before experimental activities. The results of Levene's test indicated that the assumption of homogeneity of variances was not rejected ($F=0.81$, $p=0.489>0.05$). Table 8 shows the results of the two-way ANCOVA. The results showed that the teaching methods had a significant effect on the students' critical thinking tendency ($F=5.59$, $p=0.020<0.05$), and the effect size was 0.05. There was a significant interaction between teaching methods and learning achievement in the critical thinking tendency ($F=7.48$, $p=0.007<0.01$). Furthermore, the interaction effect ($\eta 2$) between the two was 0.06, which represented a relatively small effect (Cohen, 1988).

To further explore the results of a simple main-effects analysis of students' academic achievement levels, this study used different teaching methods to examine students' critical thinking tendency. As shown in Table 9, the results showed that there were no significant differences in critical thinking tendency among students of different achievement levels in the experimental group ($F=1.18$, $p=0.283>0.05$). On the other hand, there were significant differences in critical thinking tendency among students of different academic achievement in the control group ($F=16.01$,

Table 8 The result of two-way ANCOVA of critical thinking tendency

Variables	*SS*	df	*MS*	*F*	η^2
Pre-test	8.73	1	8.73	29.04***	0.21
Learning approaches	1.68	1	1.68	5.59*	0.05
Learning achievement	0.38	1	0.38	1.29	
Learning approaches*Learning achievement	2.25	1	2.25	7.48**	0.06
Error	33.38	111	0.30		

*$p<0.05$, **$p<0.01$, ***$p<0.001$

Table 9 Simple main effect analysis results of learning achievement levels on the students' critical thinking tendency

Variables		*SS*	df	*MS*	*F*	η^2
M-CPA (experimental group)	Between groups	0.37	1	0.37	1.18	
	Within groups	15.21	48	0.32		
	Total	20.47	50			
C- CPA (control group)	Between groups	2.70	1	2.70	9.34**	0.13
	Within groups	17.90	62	0.29		
	Total	26.88	64			

***$p<0.001$

Table 10 Simple main effect analysis results of learning approach on the students' critical thinking tendency

Variables		*SS*	*df*	*MS*	*F*	η^2
Higher achievement	Between groups	0.05	1	0.05	0.18	
	Within groups	17.24	57	0.30		
	Total	24.11	59			
Lower achievement	Between groups	4.29	1	4.29	14.89***	0.22
	Within groups	15.26	53	0.29		
	Total	25.36	55			

***$p < 0.001$

$p = 0.000 < 0.05$). The results showed that the critical thinking tendency of high-achieving students was significantly higher than that of low-achieving students who studied through C-CPA.

Table 10 shows the simple main effect analysis results of learning methods on the critical thinking tendency of students with different learning achievement. The results found that low-achieving students used different collaborative programming methods to learn, and the critical thinking tendency was significantly different ($F = 14.89$, $p = 0.000 < 0.01$). Although high-achieving students learned with different learning methods, there was no significant difference in critical thinking tendency ($F = 0.18$, $p = 0.678 > 0.05$). The results showed that the M-CPA was more beneficial to low-achieving students than to high-achieving students in terms of critical thinking tendency.

Figure 5 shows that different teaching methods and levels of learning achievement have an interactive effect on the students' critical thinking tendency. As can be seen from the figure, students learn with a M-CPA with higher critical thinking tendency than with C-CPA. There were no significant differences in critical thinking tendency among high-achieving students regardless of the learning approach. However, it can be seen from the figure that high-achieving students have a lower critical thinking tendency than low-achieving students after adopting the M-CPA. However, low-achieving students showed a significant increase in critical thinking tendency when learning with the M-CPA. This result suggests that M-CPA helps low-achieving students improve their critical thinking tendency.

5.4 Metacognition tendency

Two-way ANCOVA was used to understand the interactive effects of different learning strategies and different learning achievements on the metacognition of students. Two-way ANCOVA was used to understand the interactive effects of different learning strategies and different learning achievements on the metacognition tendency of students. The dependent variable was the students' post-test scores of metacognition tendency and the covariate was the students' pre-test scores of metacognition tendency. The results of Levene's test indicated

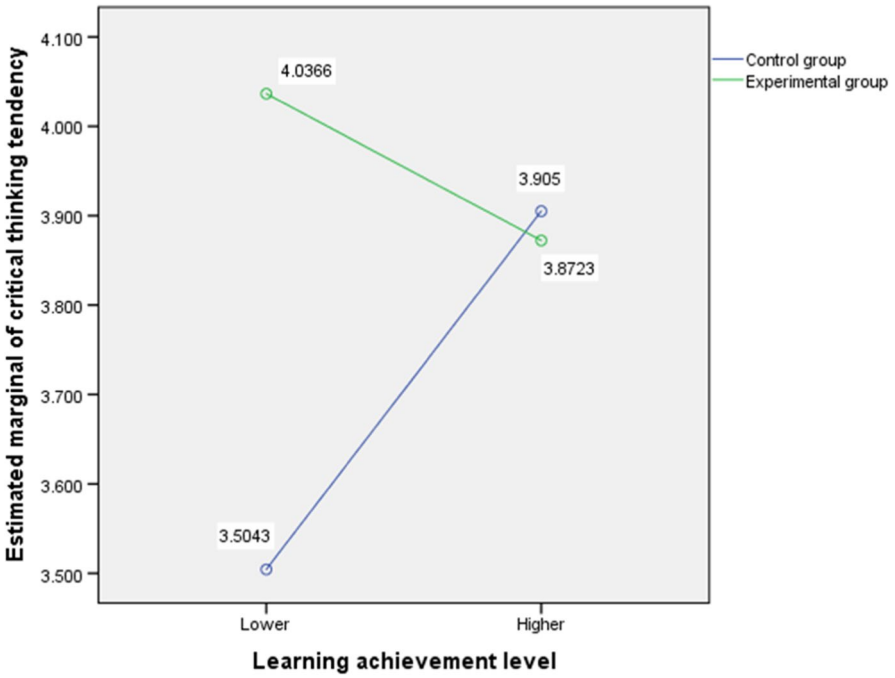

Fig. 5 Interaction in critical thinking tendency

that the assumption of the homogeneity of variances was not rejected ($F = 2.24$, $p = 0.09 > 0.05$). Table 11 shows the results of the two-way ANCOVA. The results showed that the teaching strategies had no significant effect on students' metacognition tendency (F = 2.69, $p = 0.10 > 0.05$). There was a significant interaction between teaching strategies and learning achievement in metacognition tendency ($F = 4.60$, $p = 0.03 < 0.05$). Furthermore, the interaction effect (η^2) between the two was 0.04, which represented a relatively small effect (Cohen, 1988).

To further explore the results of the simple main effects analysis of students' academic performance levels, this study used different teaching methods to examine students' metacognition tendency. As shown in Table 12, the results showed no

Table 11 The two-way ANCOVA result of metacognition tendency

Variables	*SS*	df	*MS*	*F*	η^2
Pre-test	5.80	1	5.80	13.90 ***	0.18
Learning approaches	1.12	1	1.12	2.69	
Learning achievement	0.00	1	0.00	0.00	
Learning approaches* Learning achievement	1.92	1	1.92	4.60*	0.04
Error	46.31	111	0.42		

*$p < 0.05$, ***$p < 0.001$

Table 12 Simple main effect analysis results of learning achievement levels on students' metacognition tendency

Variables		*SS*	df	*MS*	*F*
M-CPA (experimental group)	Between groups	0.78	1	0.78	1.44
	Within groups	25.91	48	0.54	
	Total	28.73	50		
C- CPA (control group)	Between groups	1.10	1	1.10	3.34
	Within groups	20.39	62	0.33	
	Total	25.83	64		

$^{*}p < 0.05$

significant differences in metacognition tendency among students with different levels of achievement in the experimental group ($F = 1.44$, $p = 0.24 > 0.05$) and the control group (F = 3.34, p = 0.07 > 0.05), which indicated that it makes no distinction in metacognitive tendency between the high-achieving students and the low-achieving students in M-CPA and C-CPA.

Table 13 showed that the results of learning methods on the metacognition of students with different levels of learning achievement levels. The results showed that there was a significant difference in metacognition tendency between low-achieving students using different collaborative programming methods ($F = 7.34$, $p = 0.009 < 0.01$). Although high-achieving students using different learning methods did not have significant differences in metacognition tendency ($F = 1.44$, $p = 0.236 > 0.05$). The results suggest that M-CPA is more beneficial to low-achieving students than to high-achieving students in metacognition tendency.

Figure 6 showed that different teaching methods and learning achievements have interactive effects on students' metacognition tendency. It can be seen that students who study with M-CPA have higher metacognition tendency than those who study with C-CPA. There was no significant difference in metacognition tendency among high-achieving students regardless of the learning programming approach they adopted. On the other hand, low-achieving students showed a significant improvement in metacognition tendency when using M-CPA. This result suggests that M-CPA helps low-achieving students improve their metacognition tendency.

Table 13 Simple main-effect analysis results of learning approach on students' metacognition tendency

Variables		*SS*	df	*MS*	*F*	η^2
Higher achievement	Between groups	0.78	1	0.78	1.44	
	Within groups	25.91	48	0.54		
	Total	28.73	50			
Lower achievement	Between groups	3.25	1	3.25	7.34**	0.12
	Within groups	23.50	53	0.44		
	Total	27.82	55			

$^{**}p < 0.01$

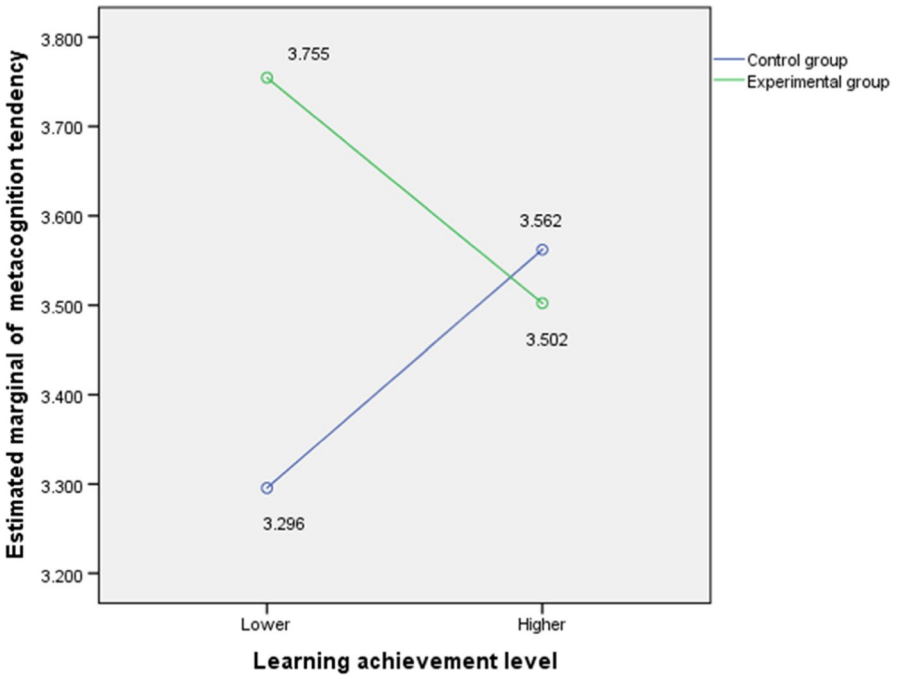

Fig. 6 Interaction in metacognition tendency

6 Discussion and conclusions

This study proposed a metacognition-based collaborative programming learning approach and explored whether teaching methods and learning achievement interact to influence computational thinking tendency, critical thinking tendency, and metacognition tendency.

The results showed that after students learned with M-CPA, there was no significant difference in basic grammar knowledge compared with students who learn with C-CPA, but there was a significant improvement in program analysis. M-CPA mainly mobilized students' metacognition through problem scaffolding to analyze, solve, and evaluate problems. Students have the opportunity to explain their understanding of the problem, discuss algorithms and procedures for the problem, clarify their ideas, and improve their understanding of the problem through discussion (Klang et al., 2021). This approach helped to improve students' ability to analyze and understand the program, and the conclusion is consistent with the conclusions of previous studies. Some researchers have proposed a flipped programming learning method that integrates self-regulation (metacognition), which significantly improves students' programming problem-solving ability (Çakıroğlu & Öztürk, 2017). They believe that self-planning and self-monitoring can help students become more aware of their learning goals and identify their problems in learning, thus helping to improved performance. Through scaffolding metacognitive questions, group members can participate more effectively in discussion and collaboration, resulting in

better learning outcomes (Barron, 2003). Zheng et al. (2022) also found that providing metacognitive regulation support to students in collaborative learning can stimulate more metacognitive behaviors and improve academic performance. However, this study also found that the collaborative programming method based on metacognition is not particularly effective for students' performance in basic grammar, and other methods need to be explored.

The results of the two-way ANCOVA show that the learning approach and learning achievement have an interactive effect on the computational thinking tendency. Compared to C-CPA, M-CPA can improve computational thinking tendencies for low- and high-achieving students. This is consistent with the conclusions of previous studies. Kramarski et al. (2002) explored the impact of metacognitively guided collaborative learning on students' real-world task-solving in mathematics. They found that students who received metacognitive instruction in collaborative learning were significantly better than those who did not and had positive effects on both high and low achievers. Moreover, the result showed that the computational thinking tendency of low achievers in the M-CPA group was even higher than that of high achievers and significantly higher than that of low achievers in the C-CPA group. For high-achieving students, there were no significant differences in computational thinking tendency between the two different learning approaches. The possible reason is that low-achieving students are often unable to actively mobilize their metacognition, and thus benefit more from metacognitive teaching (Zohar & Peled, 2008). Students improved their computational thinking by engaging in discussions guided by metacognitive questions, making connections between old and new knowledge, and exploring solutions to programming tasks, as shown in a study by Garcia (2021). Additionally, Li et al. (2022) found that metacognitive strategies can effectively aid team members in completing tasks during collaborative programming, thereby improving students' computational thinking. Therefore, for low-achieving students, teachers can provide the necessary metacognition instruction to help them mobilize metacognition, so that they can better focus on collaborative learning and solving programming problems. But for high-achieving students, since they use more metacognitive strategies in programming than novice programmers (Mohd Rum & Ismail, 2017), more effective guidance methods need to be explored.

The learning approach and learning achievement have interactive effects on students' critical thinking tendency. In the C-CPA group, the critical thinking tendency of high-achieving students were significantly higher than those of low-achieving students. On the contrary, there were no differences in critical thinking tendency among the different-achieving learners in the M-CPA group. On the other hand, high-achieving students did not show any difference in critical thinking tendency no matter which method they adopted. However, low-achieving students had a higher critical thinking tendency in M-CPA. In other words, M-CPA is more effective for critical thinking tendency of low-achieving students. Diella and Ardiansyah (2017) found that students with higher metacognition scores also scored higher on critical thinking skills. On the contrary, students with low metacognition scores also scored low on critical thinking skills. That is, metacognition is positively correlated with critical thinking. The core skills of critical thinking are interpretation, analysis, evaluation, inference, and self-regulation (Facione, 1990). Researchers have found

that open questions can develop critical thinking skills in low-achieving students. Although the critical thinking ability of low-achieving students is also relatively weak (Monrat et al., 2022), in the M-CPA group, the low-achieving students, under the guidance of metacognitive scaffolding problems, can more focus discussion on the problems solved and reflect on the process and results of cooperation, which is helpful in improving the critical thinking ability of students. In addition, the process of collaborative programming guided by metacognitive questions can improve students' understanding on learning tasks, enable them to complete tasks with clearer logic, and thus enhance their critical thinking abilities (Fanchamps et al., 2021). Previous research has also found that low-achieving learners generally rely more on structure and scaffolding than high-achieving students (Kalyuga, 2007). Therefore, low-achieving students may need metacognitive support more than high-achieving learners (Schwonke et al., 2013).

In terms of metacognition tendency, learning methods and learning achievement have interactive effects. Overall, the metacognition of the M-CPA group was better than that of the C-CPA group. The metacognition tendency of low-achieving students was significantly improved after learning through M-CPA. For high-achieving students, there was no difference in metacognition tendency regardless of the collaborative programming method used to learn. Teong (2003) found that low-achieving students showed better problem-solving skills and metacognitive behaviors after metacognitive training. During the collaborative learning process, metacognitive questions guide students to think about the learning tasks, monitor their progress, evaluate the outcomes, and reflect on what they have learned, all of which help to activate their metacognition (Ouyang et al., 2022). Therefore, metacognitive teaching can benefit low-achieving students more (Kramarski et al., 2002; Teong, 2003). This is consistent with the conclusions of this study. In addition, a person with good critical thinking ability is found to be able to carry out better metacognitive activities, especially in planning and evaluating strategies (Ku & Ho, 2010). This study found that students who studied with M-CPA showed a significant improvement in critical thinking, which also contributed to the advancement of metacognition. Since high-achieving students have strong attentional control ability (Wang et al., 2022) and have certain advantages in metacognitive regulation and control, the effect of M-CPA on high-achieving students is relatively small.

The metacognition-based collaborative programming learning method proposed in this study is more effective for low-achieving students' computational thinking tendency, critical thinking tendency, and metacognition tendency. But for high-achieving students, other ways to promote learning in collaborative programming need to be explored. In other words, different forms of metacognitive support need to be investigated for high- and low-achieving learners. Future research will provide targeted guidance based on students' learning achievement levels. Furthermore, the metacognitive support of this study is primarily based on the individual level. However, collaborative learning will involve more socially shared metacognition, which will be the direction of future research. Finally, this study collected data through tests and questionnaires. In the future, this research will continue to improve the learning system, to collect more data on the learning process to further explain the impact on students with different learning achievements.

Author contributions All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by [Wei Li], [Cheng-Ye Liu] and [Judy C.R. Tseng]. The first draft of the manuscript was written by [Wei Li] and [Judy C.R. Tseng] and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding This work was supported by National Science and Technology Council of the Republic of China [MOST 109–2511-H-216–001-MY3]; and the Ministry of Education of Humanities and Social Science Project of the People's Republic of China [21YJA880027].

Data availability The data in this study can be accessed by sending request e-mails to the corresponding author.

Declarations

Ethics approval Informed consent was obtained from all individual participants included in the study. The participants were protected by hiding their personal information during the research process. They knew that the participation was voluntary, and they could retreat at any time. All participants agree to have their data published in a journal article.

Competing interests All the authors have approved the manuscript and agree to submit it to *Education and Information Technologies*. There are no conflicts of interest to declare.

References

Akinola, S. O. (2015). Computer programming skill and gender difference: An empirical study. *American Journal of Scientific and Industrial Research, 7*(1), 1–9.

Altintas, T., Gunes, A., & Sayan, H. (2014). A peer-assisted learning experience in computer programming language learning and developing computer programming skills. *Innovations in Education and Teaching International, 53*(3), 329–337. https://doi.org/10.1080/14703297.2014.993418

Baker, L., & Brown, A. L. (1984). Metacognitive skills and reading. In P. D. Pearson, M. Kamil, R. Barr, & P. Mosenthal (Eds.), *Handbook of research in reading* (Vol. 1, pp. 353–395). Longman.

Barron, B. (2003). When Smart Groups Fail. *Journal of the Learning Sciences, 12*(3), 307–359. https://doi.org/10.1207/S15327809JLS1203_1

Binkley, M., Erstad, O., Herman, J., Raizen, S., Ripley, M., Miller-Ricci, M., & Rumble, M. (2012). Defining Twenty-First Century Skills. In P. Griffin, B. McGaw, & E. Care (Eds.), *Assessment and Teaching of 21st Century Skills* (pp. 17–66). Springer Netherlands. https://doi.org/10.1007/978-94-007-2324-5_2

Buitrago Flórez, F., Casallas, R., Hernández, M., Reyes, A., Restrepo, S., & Danies, G. (2017). Changing a Generation's Way of Thinking: Teaching Computational Thinking Through Programming. *Review of Educational Research, 87*(4), 834–860. https://doi.org/10.3102/0034654317710096

Cai, H., & Gu, X. (2019). Supporting collaborative learning using a diagram-based visible thinking tool based on cognitive load theory. *British Journal of Educational Technology, 50*(5), 2329–2345. https://doi.org/10.1111/bjet.12818

Çakıroğlu, Ü., & Öztürk, M. (2017). Flipped classroom with problem based activities-exploring self-regulated learning in a programming language course. *Educational Technology & Society*, *20*(1), 337–349.

Chai, C. S., Deng, F., Tsai, P., Koh, J. H., & Tsai, C. (2015). Assessing multidimensional students' perceptions of twenty-first-century learning practices. *Asia Pacific Education Review, 16*(3), 389–398. https://doi.org/10.1007/s12564-015-9379-4

Cho, K.-L., & Jonassen, D. H. (2002). The effects of argumentation scaffolds on argumentation and problem solving. *Educational Technology Research and Development, 50*(3), 5–22. https://doi.org/10.1007/BF02505022

Cleary, T. J., & Zimmerman, B. J. (2001). Self-regulation differences during athletic practice by experts, non-experts, and novices. *Journal of Applied Sport Psychology, 13*(2), 185–206. https://doi.org/10.1080/104132001753149883

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences*. L. Erlbaum Associates.
Demir, Ö., & Seferoglu, S. S. (2020). A Comparison of Solo and Pair Programming in Terms of Flow Experience, Coding Quality, and Coding Achievement. *Journal of Educational Computing Research, 58*(8), 1448–1466. https://doi.org/10.1177/0735633120949788
Denner, J., Werner, L., Campe, S., & Ortiz, E. (2014). Pair Programming: Under What Conditions Is It Advantageous for Middle School Students? *Journal of Research on Technology in Education, 46*(3), 277–296. https://doi.org/10.1080/15391523.2014.888272
Diella, D., & Ardiansyah, R. (2017). The correlation of metacognition with critical thinking skills of grade XI students on human excretion system concept. *Jurnal Penelitian dan Pembelajaran IPA, 3*(2), 134. https://doi.org/10.30870/jppi.v3i2.2576
DiFrancesca, D., Nietfeld, J. L., & Cao, L. (2016). A comparison of high and low achieving students on self-regulated learning variables. *Learning and Individual Differences, 45*, 228–236. https://doi.org/10.1016/j.lindif.2015.11.010
Dirkes, M. A. (1985). Metacognition: Students in charge of their thinking. *Roeper Review, 8*(2), 96–100. https://doi.org/10.1080/02783198509552944
Elia, I., Den Heuvel-Panhuizen, M., & Kolovou, A. (2009). Exploring strategy use and strategy flexibility in non-routine problem solving by primary school high achievers in mathematics. *ZDM Mathematics Education, 41*(5), 605–618. https://doi.org/10.1007/s11858-009-0184-6
Facione, P. A. (1990). *Critical thinking: A statement of expert consensus for purposes of educational assessment and instruction* (pp. 315–423). Research Findings and Recommendations. Newark, DE: American Philosophical Association. (ERIC Document Reproduction Service No. ED3155423). https://philarchive.org/archive/FACCTA. Accessed Dec 2020.
Fanchamps, N., Slangen, L., Hennissen, P., & Specht, M. (2021). The influence of SRA programming on algorithmic thinking and self-efficacy using Lego robotics in two types of instruction. *International Journal of Technology and Design Education, 31*(2), 203–222. https://doi.org/10.1007/s10798-019-09559-9
Flavell, J. H. (1976). Metacognitive aspects of problem solving. In L. B. Resnick (Ed.), *The nature of intelligence* (pp. 231–236). Erlbaum.
Flavell, J. H. (1979). Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry. *American Psychologist, 34*(10), 906–911. https://doi.org/10.1037/0003-066x.34.10.906
Flavell, J. H. (1981). Cognitive monitoring. In W. P. Dickson (Ed.), *Children's Oral Communication Skill* (pp. 35–60). Academic Press.
Garcia, M. B. (2021). Cooperative learning in computer programming: A quasi-experimental evaluation of Jigsaw teaching strategy with novice programmers. *Education and Information Technologies, 26*(4), 4839–4856.
Goel, S., & Kathuria, V. (2010). A novel approach for collaborative pair programming. *Journal of Information Technology Education: Research, 9*, 183–196. https://doi.org/10.28945/1290
Grover, S., & Pea, R. (2013). Computational thinking in K-12: A review of the state of the field. *Educational Researcher, 42*(1), 38–43. https://doi.org/10.3102/0013189x12463051
Hadwin, A. F., Järvelä, S., & Miller, M. (2011). Self-regulated, co-regulated, and socially shared regulation of learning. In *Handbook of self-regulation of learning and performance.* (pp. 65–84). Routledge/Taylor & Francis Group.
Hadwin, A., & Oshige, M. (2011). Self-regulation, coregulation, and socially shared regulation: Exploring perspectives of social in self-regulated learning theory. *Teachers College Record, 113*(2), 240–264. https://doi.org/10.1177/016146811111300204
Hanks, B., Fitzgerald, S., McCauley, R., Murphy, L., & Zander, C. (2011). Pair programming in education: A literature review. *Computer Science Education, 21*(2), 135–173. https://doi.org/10.1080/08993408.2011.579808
Hayes, J., & Stewart, I. (2016). Comparing the effects of derived relational training and computer coding on intellectual potential in school-age children. *British Journal of Educational Psychology, 86*(3), 397–411. https://doi.org/10.1111/bjep.12114
Hwang, G. J., Li, K. C., & Lai, C. L. (2020). Trends and strategies for conducting effective STEM research and applications: A mobile and ubiquitous learning perspective. *International Journal of Mobile Learning and Organisation, 14*(2), 161–183. https://doi.org/10.1504/ijmlo.2020.106166
Ireson, J., & Hallam, S. (2009). Academic self-concepts in adolescence: Relations with achievement and ability grouping in schools. *Learning and Instruction, 19*(3), 201–213. https://doi.org/10.1016/j.learninstruc.2008.04.001

Iskrenovic-Momcilovic, O. (2019). Pair programming with scratch. *Education and Information Technologies, 24*(5), 2943–2952. https://doi.org/10.1007/s10639-019-09905-3

Järvelä, S., Malmberg, J., & Koivuniemi, M. (2016). Recognizing socially shared regulation by using the temporal sequences of online chat and logs in CSCL. *Learning and Instruction, 42*, 1–11. https://doi.org/10.1016/j.learninstruc.2015.10.006

Jeong, H., & Hmelo-Silver, C. E. (2016). Seven Affordances of Computer-Supported Collaborative Learning: How to Support Collaborative Learning? How Can Technologies Help? *Educational Psychologist, 51*(2), 247–265. https://doi.org/10.1080/00461520.2016.1158654

Kafai, Y. B., & Burke, Q. (2013). Computer programming goes back to school. *Phi Delta Kappan, 95*(1), 61–65. https://doi.org/10.1177/003172171309500111

Kalyuga, S. (2007). Expertise reversal effect and its implications for learner-tailored instruction. *Educational Psychology Review, 19*(4), 509–539. https://doi.org/10.1007/s10648-007-9054-3

Kinnunen, R., & Vauras, M. (1995). Comprehension monitoring and the level of comprehension in high- and low-achieving primary school children's reading. *Learning and Instruction, 52*(2), 143–165. https://doi.org/10.1016/0959-4752(95)00009-r

Klang, N., Karlsson, N., Kilborn, W., Eriksson, P., & Karlberg, M. (2021). Mathematical problem-solving through cooperative learning—The importance of peer acceptance and friendships. *Frontiers in Education, 6*, 710296. https://doi.org/10.3389/feduc.2021.710296

Koriat, A., Ma'ayan, H., & Nussinson, R. (2006). The intricate relationships between monitoring and control in metacognition: Lessons for the cause-and-effect relation between subjective experience and behavior. *Journal of Experimental Psychology: General, 135*(1), 36–69. https://doi.org/10.1037/0096-3445.135.1.36

Kramarski, B., Mevarech, Z. R., & Lieberman, A. (2001). Effects of multilevel versus Unilevel Metacognitive training on mathematical reasoning. *The Journal of Educational Research, 94*(5), 292–300. https://doi.org/10.1080/00220670109598765

Kramarski, B., Mevarech, Z. R., & Arami, M. (2002). The effects of metacognitive instruction on solving mathematical authentic tasks. *Educational Studies in Mathematics, 49*, 225–250. https://doi.org/10.1023/A:1016282811724

Kreijns, K., Kirschner, P. A., & Jochems, W. (2003). Identifying the pitfalls for social interaction in computer-supported collaborative learning environments: A review of the research. *Computers in Human Behavior, 19*(3), 335–353. https://doi.org/10.1016/S0747-5632(02)00057-2

Ku, K. Y., & Ho, I. T. (2010). Metacognitive strategies that enhance critical thinking. *Metacognition and Learning, 5*(3), 251–267. https://doi.org/10.1007/s11409-010-9060-6

Kwon, K., Hong, R.-Y., & Laffey, J. M. (2013). The educational impact of metacognitive group coordination in computer-supported collaborative learning. *Computers in Human Behavior, 29*(4), 1271–1281. https://doi.org/10.1016/j.chb.2013.01.003

Lai, C. L. (2021). Effects of the group-regulation promotion approach on students' individual and collaborative learning performance, perceptions of regulation and regulation behaviours in project-based tasks. *British Journal of Educational Technology, 52*(6), 2278–2298. https://doi.org/10.1111/bjet.13138

Lai, C. L., & Hwang, G. J. (2014). Effects of mobile learning time on students' conception of collaboration, communication, complex problem-solving, meta-cognitive awareness and creativity. *International Journal of Mobile Learning and Organisation, 8*(3), 276–291. https://doi.org/10.1504/ijmlo.2014.067029

Law, N., Woo, D., de la Torre, J., & Wong, G. (2018). *A global framework of reference on digital literacy skills for indicator 4.4. 2.* UNESCO Institute for Statistics. Retrievedfrom http://uis.unesco.org/sites/default/files/documents/ip51-global-framework-reference-digital-literacy-skills-2018-en.pdf. Accessed Jan 2022.

Lester, F. K. (1994). Musings about mathematical problem-solving research: 1970–1994. *Journal for Research in Mathematics Education, 25*(6), 660–675. https://doi.org/10.5951/jresematheduc.25.6.0660

Li, J. S., Liu, J., Yuan, R., & Shadiev, R. (2022). The Influence of Socially Shared Regulation on Computational Thinking Performance in Cooperative Learning. *Educational Technology & Society, 25*(1), 48–60.

Lin, H., Hwang, G., & Hsu, Y. (2019). Effects of ASQ-based flipped learning on nurse practitioner learners' nursing skills, learning achievement and learning perceptions. *Computers & Education, 139*, 207–221. https://doi.org/10.1016/j.compedu.2019.05.014

Linn, M. C., & Dalbey, J. (1985). Cognitive consequences of Programming Instruction: Instruction, Access, and Ability. *Educational Psychologist, 20*(4), 191–206. https://doi.org/10.1207/s15326985ep2004_4

Lye, S. Y., & Koh, J. H. L. (2014). Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior, 41*, 51–61. https://doi.org/10.1016/j.chb.2014.09.012

Mayer, R. E. (1998). Cognitive, metacognitive, and motivational aspects of problem solving. *Instructional Science, 26*(1), 49–63.

Mayer, R. E. (2001). Cognitive, Metacognitive, and motivational aspects of problem solving. *Metacognition in Learning and Instruction*, 87–101. https://doi.org/10.1007/978-94-017-2243-8_5

McCormick, C. B. (2003). Metacognition and learning. In I. B. Weiner & D. K. Freedheim (Eds.), *Handbook of psychology, educational psychology* (pp. 79–102). John Wiley & Sons Inc.

Mohd Rum, S. N., & Ismail, M. A. (2017). Metacognitive support accelerates computer assisted learning for novice programmers. *Educational Technology & Society*, *20*(3), 170–181.

Monrat, N., Phaksunchai, M., & Chonchaiya, R. (2022). Developing students' mathematical critical thinking skills using open-ended questions and activities based on student learning preferences. *Education Research International*, 1–11. https://doi.org/10.1155/2022/3300363

Ouyang, F., Chen, S., Yang, Y., & Chen, Y. (2022). Examining the Effects of Three Group-Level Metacognitive Scaffoldings on In-Service Teachers' Knowledge Building. *Journal of Educational Computing Research, 60*(2), 352–379. https://doi.org/10.1177/07356331211030847

Panadero, E., & Järvelä, S. (2015). Socially shared regulation of learning: A review. *European Psychologist, 20*(3), 190–203. https://doi.org/10.1027/1016-9040/a000226

Papadakis, S., Kalogiannakis, M., & Zaranis, N. (2016). Developing fundamental programming concepts and computational thinking with ScratchJr in preschool education: a case study. *International Journal of Mobile Learning and Organisation*, *10*(3). https://doi.org/10.1504/ijmlo.2016.077867

Patarakin, E., Burov, V., & Yarmakhov, B. (2019). Computational pedagogy: Thinking, participation, reflection. *Digital Turn in Schools—Research, Policy, Practice*, 123–137. https://doi.org/10.1007/978-981-13-7361-9_9

Popat, S., & Starkey, L. (2019). Learning to code or coding to learn? A systematic review. *Computers & Education, 128*, 365–376. https://doi.org/10.1016/j.compedu.2018.10.005

Psycharis, S., & Kallia, M. (2017). The effects of computer programming on high school students' reasoning skills and mathematical self-efficacy and problem solving. *Instructional Science, 45*(5), 583–602. https://doi.org/10.1007/s11251-017-9421-5

Rahmat, M., Shahrani, S., Latih, R., Yatim, N. F. M., Zainal, N. F. A., & Rahman, R. A. (2012). Major Problems in Basic Programming that Influence Student Performance. *Procedia - Social and Behavioral Sciences, 59*, 287–296. https://doi.org/10.1016/j.sbspro.2012.09.277

Rao, N., Moely, B. E., & Sachs, J. (2000). Motivational beliefs, study strategies, and mathematics attainment in high- and low-achieving chinese secondary school students. *Contemporary Educational Psychology, 25*(3), 287–316. https://doi.org/10.1006/ceps.1999.1003

Robins, A., Rountree, J., & Rountree, N. (2003). Learning and teaching programming: A review and discussion. *Computer Science Education, 13*(2), 137–172. https://doi.org/10.1076/csed.13.2.137.14200

Rubio Sánchez, M., Kinnunen, P., Pareja Flores, C., & Velázquez Iturbide, Á. (2014). Student perception and usage of an automated programming assessment tool. *Computers in Human Behavior, 31*, 453–460. https://doi.org/10.1016/j.chb.2013.04.001

Scherer, R. (2016). Learning from the past–the need for empirical evidence on the transfer effects of computer programming skills. *Frontiers in Psychology, 7*, 1390. https://doi.org/10.3389/fpsyg.2016.01390

Schraw, G., & Moshman, D. (1995). Metacognitive theories. *Educational Psychology Review, 7*(4), 351–371. https://doi.org/10.1007/bf02212307

Schwonke, R., Ertelt, A., Otieno, C., Renkl, A., Aleven, V., & Salden, R. J. C. M. (2013). Metacognitive support promotes an effective use of instructional resources in intelligent tutoring. *Learning and Instruction, 23*, 136–150. https://doi.org/10.1016/j.learninstruc.2012.08.003

Seventika, S. Y., Sukestiyarno, Y. L., & Mariani, S. (2018). Critical thinking analysis based on Facione (2015) – Angelo (1995) logical mathematics material of vocational high school (VHS). *Journal of Physics: Conference Series*, *983*. https://doi.org/10.1088/1742-6596/983/1/012067

Shute, V. J., Sun, C., & Asbell-Clarke, J. (2017). Demystifying computational thinking. *Educational Research Review, 22*, 142–158. https://doi.org/10.1016/j.edurev.2017.09.003

Teong, S. (2003). The effect of metacognitive training on mathematical word-problem solving. *Journal of Computer Assisted Learning, 19*(1), 46–55. https://doi.org/10.1046/j.0266-4909.2003.00005.x

Wang, Y., Li, H., Feng, Y., Jiang, Y., & Liu, Y. (2012). Assessment of programming language learning based on peer code review model: Implementation and experience report. *Computers & Education, 59*(2), 412–422. https://doi.org/10.1016/j.compedu.2012.01.007

Wang, J., Stebbins, A., & Ferdig, R. E. (2022). Examining the effects of students' self-efficacy and prior knowledge on learning and visual behavior in a physics game. *Computers & Education*, *178*. https://doi.org/10.1016/j.compedu.2021.104405

Wei, X., Lin, L., Meng, N., Tan, W., Kong, S.-C., & Kinshuk. (2021). The effectiveness of partial pair programming on elementary school students' Computational Thinking skills and self-efficacy. *Computers & Education*, *160*. https://doi.org/10.1016/j.compedu.2020.104023

White, B. Y., & Frederiksen, J. R. (1998). Inquiry, modeling, and metacognition: Making science accessible to all students. *Cognition and Instruction, 16*(1), 3–118. https://doi.org/10.1207/s1532690xci1601_2

Wong, G.K.-W., & Cheung, H.-Y. (2018). Exploring children's perceptions of developing twenty-first century skills through computational thinking and programming. *Interactive Learning Environments, 28*(4), 438–450. https://doi.org/10.1080/10494820.2018.1534245

Zheng, L., Zhen, Y., Niu, J., & Zhong, L. (2022). An exploratory study on fade-in versus fade-out scaffolding for novice programmers in online collaborative programming settings. *Journal of Computing in Higher Education, 34*(2), 489–516. https://doi.org/10.1007/s12528-021-09307-w

Zhong, B., Wang, Q., & Chen, J. (2016). The impact of social factors on pair programming in a primary school. *Computers in Human Behavior, 64*, 423–431. https://doi.org/10.1016/j.chb.2016.07.017

Zohar, A., & Peled, B. (2008). The effects of explicit teaching of metastrategic knowledge on low- and high-achieving students. *Learning and Instruction, 18*(4), 337–353. https://doi.org/10.1016/j.learninstruc.2007.07.001

Zohar, A., Degani, A., & Vaaknin, E. (2001). Teachers' beliefs about low-achieving students and higher order thinking. *Teaching and Teacher Education, 17*(4), 469–485. https://doi.org/10.1016/s0742-051x(01)00007-5